



Reinforcement Learning

ID6002W

P Satya Jayadev

Web Enabled M.Tech,
Indian Institute of Technology Madras

Topic 1

Markov Decision Processes



1. Intro to Markov Decision Processes (MDPs)
2. Formulation of an MDP
3. Optimal Policies for MDPs
4. Value Function and Bellman Equation



Markov Decision Process

Markov Decision Process (MDP)

- ▶ MDP is a formal approach to capturing agent-environment interaction in RL
- ▶ Agent:
 - Learning and decision maker in the process
 - Characterised by the possible actions it can take
 - Goal of the agent is to maximise cumulative reward over time (or sequence)



Markov Decision Process (MDP)

- ▶ MDP is a formal approach to capturing agent-environment interaction in RL
- ▶ Agent:
 - Learning and decision maker in the process
 - Characterised by the possible actions it can take
 - Goal of the agent is to maximise cumulative reward over time (or sequence)
- ▶ Environment:
 - System with which an agent interacts
 - Applies the action decided by the agent and returns the reward
 - Characterised by states, state transitions dynamics and reward function



Markov Decision Process (MDP)

- ▶ MDP is a formal approach to capturing agent-environment interaction in RL
- ▶ **Agent:**
 - Learning and decision maker in the process
 - Characterised by the possible actions it can take
 - Goal of the agent is to maximise cumulative reward over time (or sequence)
- ▶ **Environment:**
 - System with which an agent interacts
 - Applies the action decided by the agent and returns the reward
 - Characterised by states, state transitions dynamics and reward function
- ▶ **Example:** Learning to ride a bicycle
 - Agent: Rider who is learning to ride a bicycle
 - Environment: Comprises of bicycle, road, other vehicles on the road, etc.



State of an Environment

- ▶ Snapshot or description of the environment which is relevant to decision making at a specific time
- ▶ Mathematically, this includes one or multiple variables which describe the state of the environment or system



State of an Environment

- ▶ Snapshot or description of the environment which is relevant to decision making at a specific time
- ▶ Mathematically, this includes one or multiple variables which describe the state of the environment or system
- ▶ Examples:
 1. In a maze based video game, environment state might include current location in the maze, positions of obstacles or enemies, health left, etc.
 2. In a environment where weather plays a critical role, state might include temperature, humidity level, precipitation, wind speed, etc.



State of an Environment

- ▶ Snapshot or description of the environment which is relevant to decision making at a specific time
- ▶ Mathematically, this includes one or multiple variables which describe the state of the environment or system
- ▶ **Examples:**
 1. In a maze based video game, environment state might include current location in the maze, positions of obstacles or enemies, health left, etc.
 2. In a environment where weather plays a critical role, state might include temperature, humidity level, precipitation, wind speed, etc.
- ▶ Variables to be considered in the state are decided while formulating the RL problem
- ▶ **Note:** Present state of an environment need not be fully observable (POMDP)



Some Notation

- ▶ S_t : A random variable which represents state of an environment at time t
- ▶ s : Represents a particular state of an environment
- ▶ **State space $\mathcal{S}$** : Set of all possible states of an environment i.e., $s \in \mathcal{S}$ - Discrete or Continuous
- ▶ A_t : A random variable which represents action taken by an agent at time t
- ▶ a : Represents a particular action that can be taken by an agent
- ▶ **Action space $\mathcal{A}$** : Set of all possible actions that an agent can take i.e., $a \in \mathcal{A}$ - Discrete or Continuous
- ▶ R_t and r : R_t represents reward from the environment in response to action A_t and r represents a particular reward value
- ▶ $\mathcal{R}$: Set of all possible rewards which is a finite subset of $\mathbb{R}$ i.e., $r \in \mathcal{R} \subset \mathbb{R}$



Agent-Environment Interaction in MDP

1. At each time step t , the agent observes the current state S_t of the environment



Agent-Environment Interaction in MDP

1. At each time step t , the agent observes the current state S_t of the environment
2. Agent decides and executes an action A_t , according to its policy



Agent-Environment Interaction in MDP

1. At each time step t , the agent observes the current state S_t of the environment
2. Agent decides and executes an action A_t , according to its policy
3. Agent receives an immediate reward R_{t+1} based on the reward function



Agent-Environment Interaction in MDP

1. At each time step t , the agent observes the current state S_t of the environment
2. Agent decides and executes an action A_t , according to its policy
3. Agent receives an immediate reward R_{t+1} based on the reward function
4. Environment transitions to a new state S_{t+1} according to state transition dynamics



Agent-Environment Interaction in MDP

1. At each time step t , the agent observes the current state S_t of the environment
2. Agent decides and executes an action A_t , according to its policy
3. Agent receives an immediate reward R_{t+1} based on the reward function
4. Environment transitions to a new state S_{t+1} according to state transition dynamics
5. Process repeats for subsequent time steps creating a trajectory as follows:
 $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$

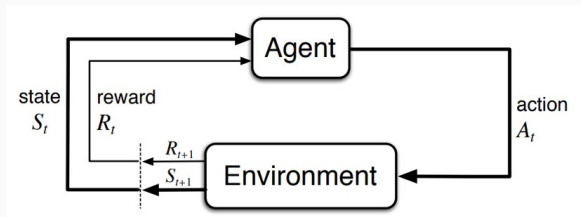


Figure 1: Agent–environment interaction in a Markov decision process.



State Transition Dynamics (Model) of an Environment

- ▶ State transition dynamics describe the change in the environment in response to actions taken by an agent i.e., they model the environment
- ▶ Mathematically, this can be a stochastic or deterministic function



State Transition Dynamics (Model) of an Environment

- ▶ State transition dynamics describe the change in the environment in response to actions taken by an agent i.e., they model the environment
- ▶ Mathematically, this can be a stochastic or deterministic function
- ▶ **Markov Property:** Next state of the environment only depends on present state and present action but not on the past
- ▶ **Deterministic State Transition:** $S_{t+1} = f(S_t, A_t)$
- ▶ **Stochastic State Transition:** $S_{t+1} = Pr(S_{t+1}|S_t, A_t)$



State Transition Dynamics (Model) of an Environment

- ▶ State transition dynamics describe the change in the environment in response to actions taken by an agent i.e., they model the environment
- ▶ Mathematically, this can be a stochastic or deterministic function
- ▶ **Markov Property:** Next state of the environment only depends on present state and present action but not on the past
- ▶ **Deterministic State Transition:** $S_{t+1} = f(S_t, A_t)$
- ▶ **Stochastic State Transition:** $S_{t+1} = Pr(S_{t+1}|S_t, A_t)$
- ▶ From an agent's perspective, understanding the state transition dynamics helps in taking best decisions to achieve the goal
- ▶ State transition function should be known while formulating the RL problem



State Transition Diagram of an MDP

- Visual representation of state transition dynamics and rewards of a finite MDP

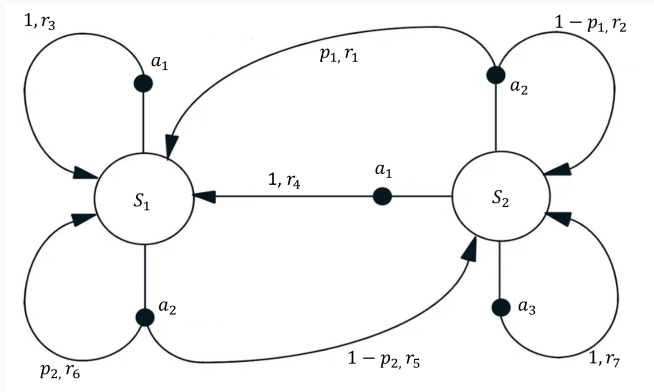


Figure 2: Example of a State Transition Diagram



Reward Function

- ▶ **Reward:** Feedback from the environment indicating how good or bad is the action taken by the agent
- ▶ Reward function gives the reward (scalar value) to the agent based on the action taken
- ▶ Reward can depend on present state, present action and next state

$$R_{t+1} = f(S_t, A_t, S_{t+1})$$



Reward Function

- ▶ **Reward:** Feedback from the environment indicating how good or bad is the action taken by the agent
- ▶ Reward function gives the reward (scalar value) to the agent based on the action taken
- ▶ Reward can depend on present state, present action and next state

$$R_{t+1} = f(S_t, A_t, S_{t+1})$$

- ▶ In an MDP, reward plays a crucial role in shaping the agent's learning and behaviour
- ▶ Reward should communicate to the agent what is to be achieved and how
- ▶ Reward function should be decided while formulating the RL problem



- ▶ Strategy or a function $\pi(s)$ that stipulates the action to be taken by an agent in a given state
- ▶ Policy function is a mapping from states to actions



- ▶ Strategy or a function $\pi(s)$ that stipulates the action to be taken by an agent in a given state
- ▶ Policy function is a mapping from states to actions
- ▶ Categorisation of policy functions:
 - **Deterministic Policy:** Gives a specific action to be taken in a given state



- ▶ Strategy or a function $\pi(s)$ that stipulates the action to be taken by an agent in a given state
- ▶ Policy function is a mapping from states to actions
- ▶ Categorisation of policy functions:
 - **Deterministic Policy:** Gives a specific action to be taken in a given state
 - **Stochastic Policy:** Assigns probabilities to different actions that can be taken in a given state



- ▶ Strategy or a function $\pi(s)$ that stipulates the action to be taken by an agent in a given state
- ▶ Policy function is a mapping from states to actions
- ▶ Categorisation of policy functions:
 - **Deterministic Policy:** Gives a specific action to be taken in a given state
 - **Stochastic Policy:** Assigns probabilities to different actions that can be taken in a given state



- ▶ Strategy or a function $\pi(s)$ that stipulates the action to be taken by an agent in a given state
- ▶ Policy function is a mapping from states to actions
- ▶ Categorisation of policy functions:
 - **Deterministic Policy:** Gives a specific action to be taken in a given state
 - **Stochastic Policy:** Assigns probabilities to different actions that can be taken in a given state
- ▶ Policy can be represented in the form of a lookup table or a parametric function



Summary of an MDP Formulation

- ▶ Agent which can take actions from a set of actions $\mathcal{A}$ based on a policy π



Summary of an MDP Formulation

- ▶ Agent which can take actions from a set of actions $\mathcal{A}$ based on a policy π
- ▶ Environment with a set of states $\mathcal{S}$ and state transition function based on which the agent interacts



Summary of an MDP Formulation

- ▶ Agent which can take actions from a set of actions $\mathcal{A}$ based on a policy π
- ▶ Environment with a set of states $\mathcal{S}$ and state transition function based on which the agent interacts
- ▶ Reward function which gives feedback to the agent on the actions taken



Summary of an MDP Formulation

- ▶ Agent which can take actions from a set of actions $\mathcal{A}$ based on a policy π
- ▶ Environment with a set of states $\mathcal{S}$ and state transition function based on which the agent interacts
- ▶ Reward function which gives feedback to the agent on the actions taken
- ▶ **Objective of an MDP:** Find an optimal policy (Control) which maximises the cumulative reward over time



Summary of an MDP Formulation

- ▶ Agent which can take actions from a set of actions $\mathcal{A}$ based on a policy π
- ▶ Environment with a set of states $\mathcal{S}$ and state transition function based on which the agent interacts
- ▶ Reward function which gives feedback to the agent on the actions taken
- ▶ **Objective of an MDP:** Find an optimal policy (Control) which maximises the cumulative reward over time
- ▶ **RL for Solving MDP:** RL algorithms enable an agent to learn an optimal policy by interacting with the environment and observing the rewards



More Notation

- ▶ Probabilistic notation is followed in general
- ▶ $\pi(a|s)$: probability of taking action a when in state s and following policy π
- ▶ $p(s', r|s, a)$: probability of transition to state s' with reward r from state s taking action a

$$p(s', r|s, a) = \text{Pr}(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a)$$

- ▶ $p(s'|s, a)$: State transition probability i.e., probability of transition to state s' from state s taking action a
- ▶ $r(s, a)$: expected reward from state s after taking action a
- ▶ $r(s, a, s')$: expected reward on transition from s to s' state s after taking action a
- ▶ **Trajectory: A sequence of states, actions and rewards**

$$\tau = S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$$



Example of an MDP

Problem Statement:

- ▶ A computer manufacturing company sells a product that can either be hot-selling, or selling poorly.



Example of an MDP

Problem Statement:

- ▶ A computer manufacturing company sells a product that can either be hot-selling, or selling poorly.
- ▶ **Hot Selling Product:**
 - Company can advertise this product to get an immediate reward of 4 units. On doing so, with probability 0.8, the product continues to sell well and otherwise, with probability of 0.2
 - The company can alternately decide not to advertise a hot-selling product yielding an immediate reward of 6 units with a 0.5 probability that the product continues to remain hot-selling.



Example of an MDP

Problem Statement:

- ▶ A computer manufacturing company sells a product that can either be hot-selling, or selling poorly.
- ▶ **Hot Selling Product:**
 - Company can advertise this product to get an immediate reward of 4 units. On doing so, with probability 0.8, the product continues to sell well and otherwise, with probability of 0.2
 - The company can alternately decide not to advertise a hot-selling product yielding an immediate reward of 6 units with a 0.5 probability that the product continues to remain hot-selling.
- ▶ **Poor Selling Product:**
 - Company can research to improve, which leads to reward of -5 units and a 0.7 probability of the research leading to the product becoming a hot-selling one.
 - Company can decide not to perform research for a poorly selling product, it receives an immediate reward of -3, with the product continuing to sell poorly with a probability of 0.6.



Example of an MDP

Problem Statement:

- ▶ A computer manufacturing company sells a product that can either be hot-selling, or selling poorly.
- ▶ **Hot Selling Product:**
 - Company can advertise this product to get an immediate reward of 4 units. On doing so, with probability 0.8, the product continues to sell well and otherwise, with probability of 0.2
 - The company can alternately decide not to advertise a hot-selling product yielding an immediate reward of 6 units with a 0.5 probability that the product continues to remain hot-selling.
- ▶ **Poor Selling Product:**
 - Company can research to improve, which leads to reward of -5 units and a 0.7 probability of the research leading to the product becoming a hot-selling one.
 - Company can decide not to perform research for a poorly selling product, it receives an immediate reward of -3, with the product continuing to sell poorly with a probability of 0.6.
- ▶ **Objective:** Formulate this problem as an MDP



Example of an MDP Contd...

- Set of Actions $\mathcal{A} = \{ \text{advertise, not-advertise, research, not-research} \}$



Example of an MDP Contd...

- ▶ Set of Actions $\mathcal{A} = \{ \text{advertise, not-advertise, research, not-research} \}$
- ▶ Set of states $\mathcal{S} = \{ \text{hot-selling, poorly-selling} \}$
- ▶ Possible actions in each state: $\mathcal{A}(\text{hot-selling}) = \{ \text{advertise, not-advertise} \}$,
 $\mathcal{A}(\text{poorly-selling}) = \{ \text{research, not-research} \}$



Example of an MDP Contd...

- ▶ Set of Actions $\mathcal{A} = \{ \text{advertise, not-advertise, research, not-research} \}$
- ▶ Set of states $\mathcal{S} = \{ \text{hot-selling, poorly-selling} \}$
- ▶ Possible actions in each state: $\mathcal{A}(\text{hot-selling}) = \{ \text{advertise, not-advertise} \}$,
 $\mathcal{A}(\text{poorly-selling}) = \{ \text{research, not-research} \}$
- ▶ Based on the states, actions and information in the problem, we can model the environment i.e., get state-transition probabilities
 - $p(\text{hot-selling} | \text{hot-selling, advertise}) =$
 - $p(\text{poorly-selling} | \text{hot-selling, advertise}) =$
 - $p(\text{hot-selling} | \text{poorly-selling, research}) =$
 - $p(\text{poorly-selling} | \text{poorly-selling, research}) =$



Learning Optimal Policies (Control)

► Episodic Tasks:

- Agent-environment interaction occurs over finite time steps with a certain start state and terminal state (finite horizon)
- Termination conditions: Pre-defined terminal state, achieving a certain goal, max. number of steps reached, etc.



Episodic and Continuing Tasks

► Episodic Tasks:

- Agent-environment interaction occurs over finite time steps with a certain start state and terminal state (finite horizon)
- Termination conditions: Pre-defined terminal state, achieving a certain goal, max. number of steps reached, etc.
- Environment resets to initial state after each episode
- Examples: Games with levels or lives, systems modelled for a fixed period of time, etc.

► Continuing Tasks:

- Agent-environment interaction continues indefinitely without any known termination (infinite or indefinite horizon)



Episodic and Continuing Tasks

► Episodic Tasks:

- Agent-environment interaction occurs over finite time steps with a certain start state and terminal state (finite horizon)
- Termination conditions: Pre-defined terminal state, achieving a certain goal, max. number of steps reached, etc.
- Environment resets to initial state after each episode
- Examples: Games with levels or lives, systems modelled for a fixed period of time, etc.

► Continuing Tasks:

- Agent-environment interaction continues indefinitely without any known termination (infinite or indefinite horizon)
- Examples: Control of systems whose performance is to be optimized over time without any end point like supply chain management, energy management, traffic management, etc.



- ▶ **Return:** Total cumulative reward that an agent receives from a particular timestep onward



- ▶ **Return:** Total cumulative reward that an agent receives from a particular timestep onward
- ▶ Let the sequence of rewards received from timestep t be

$$R_{t+1}, R_{t+2}, R_{t+3}, \dots$$

- ▶ Return denoted G_t , is given by:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

T is a final time step (terminal state of an episode).



- ▶ **Return:** Total cumulative reward that an agent receives from a particular timestep onward
- ▶ Let the sequence of rewards received from timestep t be

$$R_{t+1}, R_{t+2}, R_{t+3}, \dots$$

- ▶ Return denoted G_t , is given by:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

T is a final time step (terminal state of an episode).

- ▶ **Q:** What if it is a continuing task i.e., $T = \infty$?



- ▶ **Return:** Total cumulative reward that an agent receives from a particular timestep onward
- ▶ Let the sequence of rewards received from timestep t be

$$R_{t+1}, R_{t+2}, R_{t+3}, \dots$$

- ▶ Return denoted G_t , is given by:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

T is a final time step (terminal state of an episode).

- ▶ **Q:** What if it is a continuing task i.e., $T = \infty$? Return also goes to ∞ which is not suitable for learning an optimal policy since we want to maximise the return



Discounted Return

- ▶ Hence, we consider an approach of discounting the return



Discounted Return

- ▶ Hence, we consider an approach of discounting the return
- ▶ **Approach:** Return infinite sum of rewards, each discounted exponentially by a factor of γ (discount rate)



Discounted Return

- ▶ Hence, we consider an approach of discounting the return
- ▶ **Approach:** Return infinite sum of rewards, each discounted exponentially by a factor of γ (discount rate)
- ▶ Discounted return is defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{k=\infty} \gamma^k R_{t+k+1},$$

where $0 \leq \gamma \leq 1$



Discounted Return

- ▶ Hence, we consider an approach of discounting the return
- ▶ **Approach:** Return infinite sum of rewards, each discounted exponentially by a factor of γ (discount rate)
- ▶ Discounted return is defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{k=\infty} \gamma^k R_{t+k+1},$$

where $0 \leq \gamma \leq 1$

- ▶ Also, returns at successive timesteps are related in the following way:

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} \dots \\ G_t &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} \dots) \\ G_t &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$



Mathematical approach to finding an Optimal Policy

- Q: What is expected from an optimal policy?



Mathematical approach to finding an Optimal Policy

- ▶ Q: What is expected from an optimal policy?
- ▶ A: To give the best action to be taken in each state to maximise the expected return



Mathematical approach to finding an Optimal Policy

- ▶ Q: What is expected from an optimal policy?
- ▶ A: To give the best action to be taken in each state to maximise the expected return
- ▶ Q: How to determine the best action in each state?



Mathematical approach to finding an Optimal Policy

- ▶ Q: What is expected from an optimal policy?
- ▶ A: To give the best action to be taken in each state to maximise the expected return
- ▶ Q: How to determine the best action in each state?
- ▶ A: By evaluating the value of each action at a state in terms of expected future rewards



Mathematical approach to finding an Optimal Policy

- ▶ Q: What is expected from an optimal policy?
- ▶ A: To give the best action to be taken in each state to maximise the expected return
- ▶ Q: How to determine the best action in each state?
- ▶ A: By evaluating the value of each action at a state in terms of expected future rewards
- ▶ This is achieved by defining Value function



Mathematical approach to finding an Optimal Policy

- ▶ Q: What is expected from an optimal policy?
- ▶ A: To give the best action to be taken in each state to maximise the expected return
- ▶ Q: How to determine the best action in each state?
- ▶ A: By evaluating the value of each action at a state in terms of expected future rewards
- ▶ This is achieved by defining **Value function**
- ▶ Value functions help us order the policies of an MDP i.e., they indicate which policy is better



State Value Function

- ▶ Estimates how good it is for the agent to be in a state in terms of expected future rewards, following a policy
- ▶ Value function of a state s under a policy π is denoted $v_{\pi}(s)$



State Value Function

- ▶ Estimates how good it is for the agent to be in a state in terms of expected future rewards, following a policy
- ▶ Value function of a state s under a policy π is denoted $v_\pi(s)$
- ▶ **Definition:** Expected return that an agent gets when starting in s and following π

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

- ▶ Expectation includes the probabilistic nature of the environment as well as the reward function



State Value Function

- ▶ Estimates how good it is for the agent to be in a state in terms of expected future rewards, following a policy
- ▶ Value function of a state s under a policy π is denoted $v_\pi(s)$
- ▶ **Definition:** Expected return that an agent gets when starting in s and following π

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

- ▶ Expectation includes the probabilistic nature of the environment as well as the reward function
- ▶ **Note:** The value of the terminal state, if any, is always zero.



Action Value Function

- ▶ Estimates how good it is for the agent to take an action in a state in terms of expected future rewards, following a policy
- ▶ Value function of a state-action pair s, a under a policy π is denoted $q_{\pi}(s, a)$
- ▶ Also referred to as action value or Q-value



Action Value Function

- ▶ Estimates how good it is for the agent to take an action in a state in terms of expected future rewards, following a policy
- ▶ Value function of a state-action pair s, a under a policy π is denoted $q_\pi(s, a)$
- ▶ Also referred to as action value or Q-value
- ▶ **Definition:** Expected return that an agent gets when starting in s , taking the action a , and thereafter following policy π

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$



► Tabular Form:

- Values are stored in a table with one column for states or state-action pairs and another for their respective values
- suitable for discrete and finite state and action spaces



► Tabular Form:

- Values are stored in a table with one column for states or state-action pairs and another for their respective values
- suitable for discrete and finite state and action spaces

► Parametric Form:

- Values are approximated using a parametric function
- Function approximators can be linear, polynomial or even neural network based
- Suitable for high dimensional continuous state and action spaces



Tabular Value Function for a simple MDP

- Consider a grid $4 * 4$ world labelled from $(0,0)$ to $(3,3)$

$(0,0)$	$(0,1)$	$(0,2)$	$(0,3)$
$(1,0)$	$(1,1)$	$(1,2)$	$(1,3)$
$(2,0)$	$(2,1)$	$(2,2)$	$(2,3)$
$(3,0)$	$(3,1)$	$(3,2)$	$D : (3,3)$



Tabular Value Function for a simple MDP

- ▶ Consider a grid $4 * 4$ world labelled from $(0,0)$ to $(3,3)$
- ▶ Agent can start at any state
- ▶ Goal is to reach the destination state $(3,3)$

$(0,0)$	$(0,1)$	$(0,2)$	$(0,3)$
$(1,0)$	$(1,1)$	$(1,2)$	$(1,3)$
$(2,0)$	$(2,1)$	$(2,2)$	$(2,3)$
$(3,0)$	$(3,1)$	$(3,2)$	$D : (3,3)$



Tabular Value Function for a simple MDP

- ▶ Consider a grid $4 * 4$ world labelled from $(0,0)$ to $(3,3)$
- ▶ Agent can start at any state
- ▶ Goal is to reach the destination state $(3,3)$
- ▶ **Actions:** Agent can move in four directions: up, down, left, right

$(0,0)$	$(0,1)$	$(0,2)$	$(0,3)$
$(1,0)$	$(1,1)$	$(1,2)$	$(1,3)$
$(2,0)$	$(2,1)$	$(2,2)$	$(2,3)$
$(3,0)$	$(3,1)$	$(3,2)$	$D : (3,3)$



Tabular Value Function for a simple MDP

- ▶ Consider a grid $4 * 4$ world labelled from $(0,0)$ to $(3,3)$
- ▶ Agent can start at any state
- ▶ Goal is to reach the destination state $(3,3)$
- ▶ **Actions:** Agent can move in four directions: up, down, left, right
- ▶ **Rewards:** Each move has a reward of -1, except reaching the destination state $(3,3)$, which gives a reward of 10

$(0,0)$	$(0,1)$	$(0,2)$	$(0,3)$
$(1,0)$	$(1,1)$	$(1,2)$	$(1,3)$
$(2,0)$	$(2,1)$	$(2,2)$	$(2,3)$
$(3,0)$	$(3,1)$	$(3,2)$	$D : (3,3)$



Tabular Value Function for a simple MDP

- ▶ Consider a grid $4 * 4$ world labelled from $(0,0)$ to $(3,3)$
- ▶ Agent can start at any state
- ▶ Goal is to reach the destination state $(3,3)$
- ▶ **Actions:** Agent can move in four directions: up, down, left, right
- ▶ **Rewards:** Each move has a reward of -1, except reaching the destination state $(3,3)$, which gives a reward of 10
- ▶ **Policy:** Agent follows a deterministic policy that minimizes the number of steps to the destination

$(0,0)$	$(0,1)$	$(0,2)$	$(0,3)$
$(1,0)$	$(1,1)$	$(1,2)$	$(1,3)$
$(2,0)$	$(2,1)$	$(2,2)$	$(2,3)$
$(3,0)$	$(3,1)$	$(3,2)$	$D : (3,3)$



Tabular Value Function for a simple MDP

- ▶ Value function is the expected cumulative reward for reaching the destination state $(3, 3)$ from each state in the grid under the given policy



Tabular Value Function for a simple MDP

- ▶ Value function is the expected cumulative reward for reaching the destination state $(3, 3)$ from each state in the grid under the given policy
- ▶ Value of any state can be computed as follows:

$$V(s) = -(\text{min steps to } (3, 3) - 1) + 10$$



Tabular Value Function for a simple MDP

- Value function is the expected cumulative reward for reaching the destination state (3, 3) from each state in the grid under the given policy
- Value of any state can be computed as follows:

$$V(s) = -(\text{min steps to}(3, 3) - 1) + 10$$

State (i,j)	V(i,j)
(0, 0)	5
(0, 1)	6
(0, 2)	7
(0, 3)	8
(1, 0)	6
(1, 1)	7
(1, 2)	8
(1, 3)	9

State (i,j)	V(i,j)
(2, 0)	7
(2, 1)	8
(2, 2)	9
(2, 3)	10
(3, 0)	8
(3, 1)	9
(3, 2)	10
(3, 3)	0



Bellman equation

- ▶ **Intuition:** Gives a recursive relation between value of a state or state-action pair and values of its successor state or state-action pair
- ▶ Bellman equation shows that the value of a state can be expressed in terms of the values of its possible next states



Bellman equation

- ▶ **Intuition:** Gives a recursive relation between value of a state or state-action pair and values of its successor state or state-action pair
- ▶ Bellman equation shows that the value of a state can be expressed in terms of the values of its possible next states
- ▶ For any policy π and any state s , the Bellman equation can be derived as follows:

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a) [r + \gamma v_{\pi}(s')]$$

- ▶ Similarly Bellman equation for $q_{\pi}(s, a)$ is given by:

$$q_{\pi}(s, a) = \sum_{s'} \sum_r p(s', r|s, a) [r + \gamma \sum_{a'} \pi(a'|s') q_{\pi}(s', a')]$$



Backup Diagrams

- Illustrates how the values of a state are updated based on Bellman equation



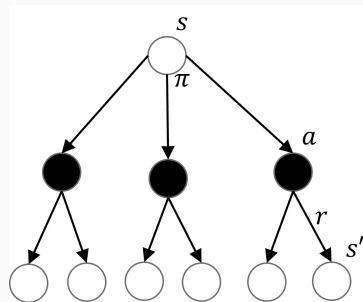
Backup Diagrams

- ▶ Illustrates how the values of a state are updated based on Bellman equation
- ▶ Nodes represent states or actions or state-action pairs
- ▶ Edges represent the transition between states or state-action pairs



Backup Diagrams

- Illustrates how the values of a state are updated based on Bellman equation
- Nodes represent states or actions or state-action pairs
- Edges represent the transition between states or state-action pairs



Optimal Value Functions

- **Comparison of Policies:** Policy π is better than or equal to policy π' if the expected return of π is greater than or equal to that of π' for all states

$$\pi \geq \pi' \text{ if and only if } v_{\pi}(s) \geq v_{\pi'}(s) \forall s \in \mathcal{S}$$



Optimal Value Functions

- **Comparison of Policies:** Policy π is better than or equal to policy π' if the expected return of π is greater than or equal to that of π' for all states

$$\pi \geq \pi' \text{ if and only if } v_{\pi}(s) \geq v_{\pi'}(s) \forall s \in \mathcal{S}$$

- **Result:** For any MDP, there exists one optimal policy π^* which is better than or equal to all other policies



Optimal Value Functions

- **Comparison of Policies:** Policy π is better than or equal to policy π' if the expected return of π is greater than or equal to that of π' for all states

$$\pi \geq \pi' \text{ if and only if } v_{\pi}(s) \geq v_{\pi'}(s) \forall s \in \mathcal{S}$$

- **Result:** For any MDP, there exists one optimal policy π^* which is better than or equal to all other policies
- State value function of an optimal policy is called Optimal state value function

$$v_*(s) = \max_{\pi} v_{\pi}(s), \forall s \in \mathcal{S}$$



Optimal Value Functions

- **Comparison of Policies:** Policy π is better than or equal to policy π' if the expected return of π is greater than or equal to that of π' for all states

$$\pi \geq \pi' \text{ if and only if } v_{\pi}(s) \geq v_{\pi'}(s) \forall s \in \mathcal{S}$$

- **Result:** For any MDP, there exists one optimal policy π^* which is better than or equal to all other policies
- State value function of an optimal policy is called Optimal state value function

$$v_*(s) = \max_{\pi} v_{\pi}(s), \forall s \in \mathcal{S}$$

- Similarly, optimal action value function is given by:

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$$



Optimal Value Functions

- **Comparison of Policies:** Policy π is better than or equal to policy π' if the expected return of π is greater than or equal to that of π' for all states

$$\pi \geq \pi' \text{ if and only if } v_{\pi}(s) \geq v_{\pi'}(s) \forall s \in \mathcal{S}$$

- **Result:** For any MDP, there exists one optimal policy π^* which is better than or equal to all other policies
- State value function of an optimal policy is called Optimal state value function

$$v_*(s) = \max_{\pi} v_{\pi}(s), \forall s \in \mathcal{S}$$

- Similarly, optimal action value function is given by:

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$$

- Optimal state and action values functions are related as follows:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a)$$



Finding an Optimal Policy

- An optimal policy is known once the optimal action value function is computed

$$\pi_*(a|s) = \arg \max_a q_*(s, a)$$

- **Note:** Every MDP will have atleast one deterministic optimal policy



Finding an Optimal Policy

- ▶ An optimal policy is known once the optimal action value function is computed

$$\pi_*(a|s) = \arg \max_a q_*(s, a)$$

- ▶ **Note:** Every MDP will have atleast one deterministic optimal policy
- ▶ Since bellman equation holds for any value function, it can be written for v_* and q_* as well, without reference to any policy
- ▶ Referred to as **Bellman Optimality Equations**



Bellman Optimality Equation

- Optimality equation expresses the fact that the value of a state under an optimal policy π^* must equal the expected return for the best action from that state



Bellman Optimality Equation

- ▶ Optimality equation expresses the fact that the value of a state under an optimal policy π^* must equal the expected return for the best action from that state
- ▶ Bellman optimality equation for v_* is derived as follows:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) = \max_a \mathbb{E}_{\pi^*} [G_t | S_t = s, A_t = a]$$



Bellman Optimality Equation

- ▶ Optimality equation expresses the fact that the value of a state under an optimal policy π^* must equal the expected return for the best action from that state
- ▶ Bellman optimality equation for v_* is derived as follows:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) = \max_a \mathbb{E}_{\pi^*} [G_t | S_t = s, A_t = a]$$

$$v_*(s) = \max_a \mathbb{E}_{\pi^*} [R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a]$$



Bellman Optimality Equation

- ▶ Optimality equation expresses the fact that the value of a state under an optimal policy π^* must equal the expected return for the best action from that state
- ▶ Bellman optimality equation for v_* is derived as follows:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) = \max_a \mathbb{E}_{\pi^*} [G_t | S_t = s, A_t = a]$$

$$v_*(s) = \max_a \mathbb{E}_{\pi^*} [R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a]$$

$$v_*(s) = \max_a \mathbb{E} [R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]$$



Bellman Optimality Equation

- ▶ Optimality equation expresses the fact that the value of a state under an optimal policy π^* must equal the expected return for the best action from that state
- ▶ Bellman optimality equation for v_* is derived as follows:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) = \max_a \mathbb{E}_{\pi^*} [G_t | S_t = s, A_t = a]$$

$$v_*(s) = \max_a \mathbb{E}_{\pi^*} [R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a]$$

$$v_*(s) = \max_a \mathbb{E} [R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]$$

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]$$



Bellman Optimality Equation

- Optimality equation expresses the fact that the value of a state under an optimal policy π^* must equal the expected return for the best action from that state
- Bellman optimality equation for v_* is derived as follows:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) = \max_a \mathbb{E}_{\pi^*} [G_t | S_t = s, A_t = a]$$

$$v_*(s) = \max_a \mathbb{E}_{\pi^*} [R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a]$$

$$v_*(s) = \max_a \mathbb{E} [R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]$$

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]$$

- Similarly, Bellman optimality equation for q_* is:

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')]$$



Finding an Optimal Value Function and an Optimal Policy

- **Result:** For a standard discounted finite MDP, the Bellman optimality equation has a unique solution for the optimal value function.



Finding an Optimal Value Function and an Optimal Policy

- ▶ **Result:** For a standard discounted finite MDP, the Bellman optimality equation has a unique solution for the optimal value function.
- ▶ Once v_* is known, an optimal policy can be obtained by choosing actions that achieve the maximum expected return:

$$\pi_*(s) \in \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]$$



Finding an Optimal Value Function and an Optimal Policy

- ▶ **Result:** For a standard discounted finite MDP, the Bellman optimality equation has a unique solution for the optimal value function.
- ▶ Once v_* is known, an optimal policy can be obtained by choosing actions that achieve the maximum expected return:

$$\pi_*(s) \in \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]$$

- ▶ For a fixed policy, Bellman expectation equations form a system of linear equations and can be solved.
- ▶ For optimality, the max operator makes the Bellman optimality equation nonlinear and makes it hard to solve.
- ▶ However, solving the bellman equations is tedious, computationally intensive and requires the state transition model



Finding an Optimal Value Function and an Optimal Policy

- ▶ **Result:** For a standard discounted finite MDP, the Bellman optimality equation has a unique solution for the optimal value function.
- ▶ Once v_* is known, an optimal policy can be obtained by choosing actions that achieve the maximum expected return:

$$\pi_*(s) \in \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]$$

- ▶ For a fixed policy, Bellman expectation equations form a system of linear equations and can be solved.
- ▶ For optimality, the max operator makes the Bellman optimality equation nonlinear and makes it hard to solve.
- ▶ However, solving the bellman equations is tedious, computationally intensive and requires the state transition model
- ▶ Therefore, RL methods often use iterative and sample-based approaches to learn optimal or near-optimal value functions and policies.



RL Methods to Learning Near-Optimal Policies

1. Monte Carlo Methods
2. Temporal Difference Learning
3. Policy Gradient Methods
4. Actor-Critic Methods
5. Deep RL



- ▶ MDP is used to formulate an sequential interaction between an agent and an environment



Summary

- ▶ MDP is used to formulate an sequential interaction between an agent and an environment
- ▶ MDP comprises of states, actions, state transition dynamics (model), reward function



Summary

- ▶ MDP is used to formulate an sequential interaction between an agent and an environment
- ▶ MDP comprises of states, actions, state transition dynamics (model), reward function
- ▶ Objective of an MDP is to find an optimal policy which gives best action to be taken in any given state to maximise return



Summary

- ▶ MDP is used to formulate an sequential interaction between an agent and an environment
- ▶ MDP comprises of states, actions, state transition dynamics (model), reward function
- ▶ Objective of an MDP is to find an optimal policy which gives best action to be taken in any given state to maximise return
- ▶ Value functions enable a mathematical evaluation and comparison of states, state-action pairs and policies



Summary

- ▶ MDP is used to formulate an sequential interaction between an agent and an environment
- ▶ MDP comprises of states, actions, state transition dynamics (model), reward function
- ▶ Objective of an MDP is to find an optimal policy which gives best action to be taken in any given state to maximise return
- ▶ Value functions enable a mathematical evaluation and comparison of states, state-action pairs and policies
- ▶ Value function is the expected return that an agent gets from a state or state-action pair



Summary

- ▶ MDP is used to formulate an sequential interaction between an agent and an environment
- ▶ MDP comprises of states, actions, state transition dynamics (model), reward function
- ▶ Objective of an MDP is to find an optimal policy which gives best action to be taken in any given state to maximise return
- ▶ Value functions enable a mathematical evaluation and comparison of states, state-action pairs and policies
- ▶ Value function is the expected return that an agent gets from a state or state-action pair
- ▶ Bellman equation gives a recursive relation between value of one state (state-action pair) to values of its successive states (state-action pairs)



Summary

- ▶ MDP is used to formulate an sequential interaction between an agent and an environment
- ▶ MDP comprises of states, actions, state transition dynamics (model), reward function
- ▶ Objective of an MDP is to find an optimal policy which gives best action to be taken in any given state to maximise return
- ▶ Value functions enable a mathematical evaluation and comparison of states, state-action pairs and policies
- ▶ Value function is the expected return that an agent gets from a state or state-action pair
- ▶ Bellman equation gives a recursive relation between value of one state (state-action pair) to values of its successive states (state-action pairs)
- ▶ Solving the Bellman optimality equation gives the optimal value function and optimal policy

